

看 Linux 内核源码时被满天的结构体嵌套、函数指针和回调函数绕晕，是非常正常的现象。Linux 内核虽然是用纯 C 语言（一种面向过程的语言）编写的，但其底层架构——特别是设备驱动模型（Driver Model）、字符设备（Character Devices）和虚拟文件系统（VFS）——**重度依赖面向对象编程（OOP）的设计思想**。

C 语言里没有 `class`、`public`、`extends` 或 `virtual` 这些关键字。在 C 语言中，面向对象不是一种语法，而是一种**设计模式**。内核开发者通过巧妙地使用 `struct`、函数指针和内存管理，手动实现了 OOP 的三大核心：**封装、继承和多态**。

这是一份为你量身定制的 C 语言面向对象教案。我们将结合实际的底层硬件和驱动开发场景，一步一步把它拆解透彻。

第一课：面向对象在 C 语言中的基石

1. 封装 (Encapsulation): 数据与行为的结合

在面向过程编程中，数据（变量）和操作数据的行为（函数）是分离的。而在 OOP 中，我们希望把它们绑定在一起，形成一个“对象”。在 C 语言中，我们用 **包含函数指针的结构体（`struct`）** 来实现封装。

内核场景模拟： 假设我们正在编写一个温湿度传感器（例如 DHT11）的底层控制代码。

```
1 //1.定义对象的的的结构体(相当于Class)
2 struct temp_sensor{
3     //属性（数据）
4     int gpio_pin;
5     int current_temp;
6
7     //方法（行为：使用函数指针）
8     //注意：必须把自身的指针作为第一个参数传进去，这相当于C++/Java里面的'this'指针
9     int (*read_data)(struct temp_sensor *self);
10    void (*set_pin)(struct temp_sensor *self,int pin);
11 };
12
13 //2.实现具体的方法
14 int dht11_read_data(struct temp_sensor *self){
15     //通过self指针访问对象自己的数据
16     int pin = self->gpio_pin;
17     // ... 执行 GPIO 电平读取和时序解析 ...
18     self->current_temp=25;//假设读到了25度
19     return self->current_temp;
20 }
21
22 void dht11_set_pin(struct temp_sensor *self,int pin){
23     self->gpio_pin=pin;
24 }
25
26 //构造函数（实例化对象）
27 struct temp_sensor* temp_sensor_create(int pin){
28     struct temp_sensor *sensor = kmalloc(sizeof(struct temp_sensor),GFP<KERNEL);
29     if(!sensor) return NULL;
30
31     sensor->gpio_pin=pin;
```

```

32     sensor->current_temp=0;
33
34     //绑定方法（将函数地址赋给函数指针）
35     sensor->read_data=dht11_read_data;
36     sensor->set_pin=dht11_set_pin;
37
38     return sensor;
39 }

```

1. 钥匙与房子（指针与属性）

当你把 `struct temp_sensor *self` 作为一个参数传给函数时，你传递的其实只是一把“钥匙”（内存地址）。

- **参数 (Parameter)**：你只交出了 **1 把钥匙**（所以它算作 1 个参数）。
- **属性 (Properties)**：这把钥匙可以打开一栋房子（也就是 `temp_sensor` 对象），房子里面确实有两样东西：`gpio_pin` 和 `current_temp`。

如果你不传这把钥匙（`self` 指针），而是把里面的属性拆开来传，那代码就会变成这样：

```

1 // 这样写，函数就有 3 个参数了，但它就失去了“面向对象”的意义
2 void set_pin_bad(int old_gpio_pin, int old_current_temp, int new_pin);

```

2. 为什么只传 `self` 一个参数就够了？

因为只要函数手里有了这把钥匙（`self` 指针），它就可以进到房子里面，随意查看或者修改那两个属性！

让我们看看真实的 `set_pin` 函数内部是怎么工作的：

```

1 // 定义真实的设置引脚函数
2 // 参数 1: self (对象的钥匙)
3 // 参数 2: pin (你要设置的新号码，比如 5)
4 void hardware_set_pin(struct temp_sensor *self, int pin) {
5
6     // 函数通过 self 这把钥匙，进入对象内部，找到了 gpio_pin 这个属性
7     // 然后把外面传进来的参数 pin 赋值给它
8     self->gpio_pin = pin;
9
10    printf("引脚已经被修改为: %d\n", self->gpio_pin);
11 }

```

总结你的发现

你的直觉非常准：虽然 `read_data` 表面上只有 `self` 这 1 个参数，但它通过这 1 个参数，间接地获得了操作那 2 个属性的权力。

这就好像我交给你一个“背包”（1 个参数）。虽然我只给了你一件东西，但你打开背包后，能看到里面有一台电脑、一个水杯（属性）。我不需要把电脑和水杯单独递给你，只要把背包给你，里面的东西就都归你管了。

这就是为什么在结构体里使用函数指针时，永远要把 `self` 作为第一个参数传进去——为了让函数知道，它该去操作哪一个“背包”里的属性。

```

1  #include <stdio.h>
2
3  //1.你的结构体定义
4  struct temp_sensor{
5      int gpio_pin;
6      int current_temp;
7
8      //函数指针(流出的坑位)
9      int (*read_temp)(struct temp_sensor *self);
10 };
11
12
13 //2.写一个真正的、具体的函数来实现读取逻辑
14 // 注意: 它的返回值和参数, 必须和结构体里的要求一模一样
15 int hardware_read_temp(struct temp_sensor *self){
16     printf("正在从GPIO针脚%d读取温度...\n", self->gpio_pin);
17     self->current_temp=25; //模拟读取到的温度值
18     return self->current_temp;
19 }
20
21 int main(){
22     //创建一个传感器“对象”
23     struct temp_sensor sensorA;
24     sensorA.gpio_pin=4;
25
26     sensorA.read_temp=hardware_read_temp; //把函数指针指向具体的实现函数
27
28     int temp=sensorA.read_temp(&sensorA); //通过函数指针调用函数
29     printf("读取到的温度是: %d\n", temp);
30     return 0;
31 }

```

2. 继承 (Inheritance): 结构体的嵌套与 `container_of`

继承的本质是“代码复用”和“Is-A (是一个)”关系。内核中有一个通用的字符设备结构体 `cdev`，所有的具体字符设备（如我们写的传感器驱动）都是一个 `cdev`。C 语言通过结构体嵌套（组合）来实现继承。

在 C++ 或 Java 里，写一句 `class MySensor extends CDev`，编译器就在底层帮你搞定了继承。但在 C 语言里，我们只能靠内存排布来模拟。

当你把基类 `struct cdev` 作为一个变量放到 `struct my_sensor_dev` 里面时，在内存中它们的布局是这样的：

```

1  [ 内存起始地址 ] ----> =====
2                                | struct cdev char_dev;          | <-- 这是一个通用的字符设
   备结构体
3                                |   - dev_t dev;                | (包含了内核需要的通
   用属性)
4                                |   - struct file_operations *ops; |
5                                |   - ...                        |
6                                =====
7                                | int gpio_pin;                  | <-- 这是你加的私有属性
8                                =====
9                                | char serial_number[16];         | <-- 这是你加的私有属性
10 [ 内存结束地址 ] =====

```

继承的本质就是“全盘接收”。由于你的 `my_sensor_dev` 把 `cdev` 整个吞了进去，所以 `my_sensor_dev` 在物理内存上确实确实包含了一个完整的 `cdev`。它拥有了 `cdev` 的所有特征，这就是“Is-A（是一个）”的关系。

```

1  #include <linux/cdev.h>
2
3  // 基类 (Base Class) 是 struct cdev, 内核已经定义好了
4
5  // 子类 (Derived Class): 我们自己的传感器设备
6  struct my_sensor_dev {
7      struct cdev char_dev; // 【继承】：将基类作为自己的第一个成员
8
9      int gpio_pin;         // 子类的私有属性
10     char serial_number[16];
11 };

```

核心魔法：`container_of` 宏 在内核中，VFS（虚拟文件系统）只认基类 `cdev`，它在调用你的驱动时，传给你的指针往往是 `struct cdev *`。但你需要操作你的子类 `my_sensor_dev` 里的 `gpio_pin`，怎么办？

内核提供了神级宏 `container_of`。它的作用是：“根据结构体内部某个成员的指针，反推出整个外部结构体的首地址。”

```

1  // 假设内核传给你的是基类的指针 cdev_ptr
2  struct cdev *cdev_ptr = ...;
3
4  // 你需要向下转型 (Downcasting) 获取子类对象:
5  struct my_sensor_dev *my_dev = container_of(cdev_ptr, struct my_sensor_dev, char_dev);
6
7  // 现在你就可以操作具体的引脚了
8  int current_pin = my_dev->gpio_pin;

```

为什么需要 `container_of` 宏？（现实困境）

假设现在有一个人来查水表（内核虚拟文件系统 VFS），他只懂标准的“水表”（`cdev`），他根本不知道，也不关心你这是一个“智能传感器”还是一个“普通LED灯”。

当硬件发生中断，或者用户敲下代码读取数据时，内核只会把通用部分（`cdev`）的指针扔回给你自己的驱动函数。

 此时你面临的困境是：

1. 内核传给你的是：`struct cdev *cdev_ptr`。
2. 但你需要操作的是引脚：`gpio_pin`。
3. `cdev` 结构体里面根本没有 `gpio_pin` 这个字段！它在外部的 `my_sensor_dev` 里面！

你需要一种方法，能顺着这根藤（`cdev`），摸到那个瓜（`my_sensor_dev` 的首地址）。

container_of 的原理解密（它是怎么反推的？）

`container_of(ptr, type, member)` 这个宏的作用就是“盲人摸象”里的逆向操作。

我们可以用一个“酒店与保险箱”的比喻来理解它的工作原理：

- 大结构体（`struct my_sensor_dev`）= 一整间酒店客房。
- 小结构体（`char_dev`）= 客房里的一个嵌入式保险箱。
- 内核传给你的指针（`cdev_ptr`）= 内核只知道这个保险箱的绝对 GPS 坐标。

`container_of` 的解题思路非常暴力且极其聪明：

1. 既然所有的客房（`my_sensor_dev`）的图纸都是一样的。
2. 那么保险箱（`char_dev`）距离客房大门（大结构体首地址）的相对距离（Offset / 偏移量）永远是固定的！
3. 假设根据图纸，保险箱距离大门永远往里走 10 米。
4. 现在内核告诉我保险箱在坐标 1010。
5. 那大门的坐标不就是 $1010 - 10 = 1000$ 吗！

在 C 语言的底层，`container_of` 就是做了这道数学题：大结构体地址 = 小结构体地址 - 小结构体在图纸里的偏移量（Offset）

这三个参数是解开 `container_of` 魔法的“三把钥匙”。

我们结合你刚才的代码示例 `container_of(cdev_ptr, struct my_sensor_dev, char_dev);`，继续用“酒店客房（外层大结构体）”和“保险箱（内层小结构体）”的比喻来逐一拆解：

1. ptr (Pointer - 指针)

- 大白话：你当前手里拿到的、已知的那个内部小结构体的内存地址。
- 比喻：现实中保险箱的绝对 GPS 坐标（比如：内核传给你的地址是 `0x1020`）。
- 对应代码：`cdev_ptr`
- 作用：这是计算的起点。内核通过参数把这个地址传给你，你要拿着这个起始坐标，往前倒推大门的坐标。

2. type (Type - 类型)

- 大白话：你想要反推出来的那个外部大结构体的类型。
- 比喻：酒店客房的设计图纸（指明了是“标准间”还是“总统套房”）。

- 对应代码： `struct my_sensor_dev`
- 作用： 编译器需要这张图纸。只有知道了外部结构体是什么类型，编译器才能在图纸上查找内部小结构体到底放在了什么位置。最后返回的结果也会被强转成这个类型（即返回 `struct my_sensor_dev *`）。

3. member (Member Name - 成员名称)

- 大白话： 内部小结构体在外部大结构体里的变量名字。
- 比喻： 图纸上标明的“保险箱”这个部件的名字。
- 对应代码： `char_dev`
- 作用： 配合第 2 个参数使用的。编译器拿到图纸（`type`）后，要在图纸上寻找叫这个名字（`member`）的部件，从而计算出它距离大门有多远（也就是算出 **Offset 偏移量**）。

💡 连起来看：它是如何计算的？

当我们写下这行代码时：

```
1 struct my_sensor_dev *my_dev = container_of(cdev_ptr, struct my_sensor_dev, char_dev);
```

编译器在底层的心智模型是这样运作的：

1. 看图纸找距离： 编译器看了看 `struct my_sensor_dev`（参数2）的定义，发现名叫 `char_dev`（参数3）的成员，正好放在结构体的最开头（偏移量为 0 字节）。
2. 拿起点做减法： 编译器拿到了现在的地址 `cdev_ptr`（参数1），然后减去刚才算出来的偏移量 0。
3. 得出终点地址： `当前地址 - 0 = 大结构体首地址`。

3. 多态 (Polymorphism)：统一接口，各自实现

多态是 Linux 内核面向对象最精髓的地方。多态的意思是：同一个接口，不同对象表现出不同的行为。

想象你手里有一个“万能遥控器”，上面只有一个按钮叫 `[播放 (Play)]`。

- 当你把它对准电视按下 `[播放]`，它开始播放画面。
- 当你把它对准音响按下 `[播放]`，它开始播放音乐。
- 当你把它对准空调按下 `[播放]`，毫无反应（因为空调不支持这个功能）。
- 在这个场景里：
 - 接口 (Interface)：就是那个 `[播放]` 按钮。你作为使用者，根本不需要管电视内部是液晶面板还是显像管，也不需要管音响是蓝牙的还是有线的。你只会按 `[播放]`。
 - 多态 (Polymorphism)：按同一个按钮，引发了完全不同的硬件动作。

在内核中，多态通过函数指针表 (Virtual Method Table, Vtable) 实现。最经典的例子就是 `struct file_operations`。

Linux 的哲学是“一切皆文件”。应用层调用 `read()`，内核怎么知道是去读硬件寄存器，还是去读磁盘文件？

```
1 struct file_operations {
2     struct module *owner;
3     ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
```

```

4     ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
5     // ... 其他几十个函数指针
6     //注意，内核只定义了指针（留了坑位），没有写任何具体的实现。
7 };
8
9 // 【具体实现 A】：传感器驱动
10 ssize_t sensor_read(struct file *filp, char __user *buf, size_t count, loff_t *f_pos)
11 {
12     // 读取 GPIO 的逻辑
13     return bytes_read;
14 }
15
16 // 实例化传感器的操作表
17 struct file_operations sensor_fops = {
18     .owner = THIS_MODULE,
19     .read = sensor_read,      // 重写接口
20 };
21
22 // ===== //
23 // 内核的 vfs 层代码（调用方）根本不关心你是传感器还是磁盘：
24 // 它只需要拿到 fops 指针，直接调用，这就是多态！
25 // file->f_op->read(file, buf, count, pos);
26 // 内核定义的“接口”（纯虚函数表）

```

理解要点： 当你在写驱动时，填充 `file_operations` 结构体，本质上就是在重写（Override）父类的虚函数，让内核在特定的时机回调你的代码。

总结例子：

```

1  #include <stdio.h>
2  #include <stddef.h> //为了使用offsetof宏
3
4  // =====
5  // 第一部分：模拟 Linux 内核的基础设施（基类、接口和神级宏）
6  // =====
7
8  // 1. 【核心魔法】定义 container_of 宏
9  // 1. 【核心魔法】定义 container_of 宏
10 #define container_of(ptr, type, member) ({ \
11     const typeof( ((type *)0)->member ) *__mptr = (ptr); \
12     (type *) ( (char *)__mptr - offsetof(type,member) ); })
13
14 // 2. 模拟内核的字符设备基类（只包含最基础的信息）
15 struct cdev {
16     int device_id;
17 };
18
19 // 3. 模拟内核的 vfs 层文件结构体（它负责携带基类指针）
20 struct file {
21     struct cdev *private_data;

```



```

22 };
23
24 // 4. 【多态接口】模拟内核的纯虚函数表 (Vtable)
25 struct file_operations {
26     int (*read)(struct file *filp);
27     int (*write)(struct file *filp, int data);
28 };
29
30 // 1. 【封装与继承】：定义我们自己的传感器对象
31 struct my_sensor {
32     struct cdev core_dev; // 【继承】：把内核基类作为第一个成员吞进去
33     int gpio_pin;         // 自己的私有属性：引脚
34     int temperature;      // 自己的私有属性：当前温度
35 };
36
37 // 2. 【多态实现】：重写具体的硬件读取逻辑
38 int sensor_read(struct file *filp) {
39     // 步骤 A：内核只会传给我们基类的指针
40     struct cdev *cdev_ptr = filp->private_data;
41
42     // 步骤 B：【神级宏出场】盲人摸象的逆向操作，顺着基类推算出整个子类对象的首地址
43     struct my_sensor *sensor = container_of(cdev_ptr, struct my_sensor, core_dev);
44
45     // 步骤 C：现在我们拥有了操作自己私有属性的权力！
46     printf("    [底层硬件] 正在激活 GPIO %d 引脚...\n", sensor->gpio_pin);
47     printf("    [底层硬件] 传感器数据读取成功! \n");
48
49     return sensor->temperature;
50 }
51
52 // 3. 【多态绑定】：把你的实现填进内核的虚函数表里
53 struct file_operations sensor_fops = {
54     .read = sensor_read, // 把通用的 read 接口，绑定到你写的 sensor_read
55     // 不写 write, 就代表这个设备不支持写入
56 };
57
58 int main() {
59     printf("--- 系统启动，初始化传感器 ---\n");
60
61     // 1. 实例化一个具体的传感器对象（比如一个接在 4 号引脚，当前温度 26 度的 DHT11 传感器）
62     struct my_sensor dht11 = {
63         .core_dev = { .device_id = 1001 },
64         .gpio_pin = 4,
65         .temperature = 26
66     };
67
68     // 2. 模拟内核准备 VFS 环境
69     // 当用户打开设备时，内核会创建一个 file 结构体，并把底层基类的指针塞进去
70     struct file mock_file;
71     mock_file.private_data = &dht11.core_dev; // 内核只认识 cdev! 它不知道 dht11 的存在
72
73     printf("\n--- 用户层发起读取请求 ---\n");

```

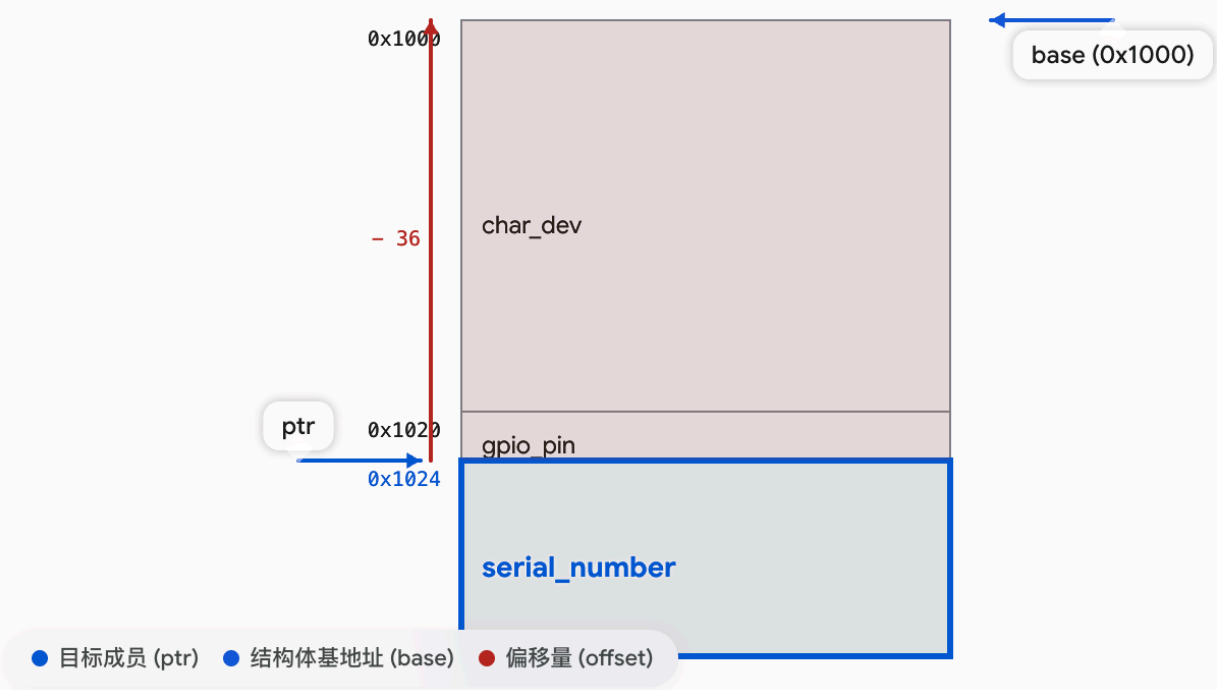


```

74
75 // 3. 【高潮：多态的发生！】
76 // 用户层调用 read(), 内核 VFS 拿着 file_operations 直接盲调!
77 // 此时内核根本不关心这是什么设备，只需要一行代码无脑执行：
78 int result = sensor_fops.read(&mock_file);
79
80 printf("\n--- 返回结果 ---\n");
81 printf(" [用户层] 最终读到的温度数据是： %d 摄氏度\n", result);
82
83 return 0;
84 }

```

container_of 宏计算原理



计算公式

$$0x1024 \text{ (ptr)} - 36 \text{ (offset)} = 0x1000 \text{ (base)}$$